

28 Forecasting Hierarchical Daily Sales

Cordero Zayas José Iván Alejandro

Módulo III: Técnicas cognitivas e introducción a Big Data
Diplomado de Ciencia de Datos
FES Acatlán

Contents

1	Introducción	2
1.1	Contexto	2
1.2	Importancia del forecasting en retail	2
1.3	Presentación del dataset (M5)	3
1.3.1	Archivo: <code>calendar.csv</code>	3
1.3.2	Archivo: <code>sell_prices.csv</code>	4
1.3.3	Archivo: <code>sales_train.csv</code>	4
1.4	Objetivo del proyecto	4
2	Descripción del problema	5
2.1	Exigencia Computacional	5
2.2	Naturaleza del Problema	5
2.2.1	Enfoque con XGBoost	5
2.2.2	Enfoque con LSTM	6
2.2.3	Transformación de datos: de tabular a secuencial	6
2.2.4	Comparación de enfoques	7
2.3	Nivel de granularidad	7
2.4	Análisis Exploratorio de Datos (EDA)	8
2.5	Feature Engineering	11
2.5.1	Ingeniería de características para XGBoost	11
2.5.2	Ingeniería de características para LSTM	11
2.5.3	Comparación entre enfoques	12
3	Modelos	13
3.1	Justificación	13
3.2	Modelo basado en XGBoost	13
3.3	Modelo basado en LSTM	14
3.4	Evaluación	15
3.5	Resultados	16
4	Conclusiones	17
5	Referencias	18

1 Introducción

1.1 Contexto

En el marco del diplomado, se propone el desarrollo de un proyecto integrador que permita aplicar de manera práctica los conceptos abordados a lo largo de los distintos módulos. Para este propósito, se seleccionó como caso de estudio la competencia M5 Forecasting Competition organizada por Kaggle, la cual constituye uno de los benchmarks más relevantes en problemas de pronóstico de demanda en retail.

El objetivo principal de esta competencia consiste en predecir la demanda diaria de 3,049 productos distribuidos en múltiples tiendas ubicadas en tres estados de Estados Unidos: Wisconsin, Texas y California. Este problema presenta desafíos significativos tanto desde el punto de vista metodológico como computacional, debido a su naturaleza jerárquica, alta dimensionalidad y comportamiento temporal complejo.

En este proyecto se abordan las etapas fundamentales de un flujo de trabajo en ciencia de datos, incluyendo la manipulación, exploración y visualización de datos. Asimismo, se desarrollan tres enfoques de modelado con el fin de comparar distintas arquitecturas de aprendizaje supervisado aplicadas a series de tiempo. Complementariamente, se implementan diversas técnicas de feature engineering orientadas a capturar patrones temporales, efectos de calendario y relaciones entre productos, con el objetivo de mejorar la capacidad predictiva de los modelos.

El código completo utilizado en este proyecto se encuentra disponible en el siguiente repositorio: https://github.com/AlephZayas/M5_Forecasting_XGBoost-LSTM.git

1.2 Importancia del forecasting en retail

El comportamiento del consumidor ha evolucionado de manera significativa tanto en canales físicos como digitales, lo que introduce un alto grado de incertidumbre para los minoristas respecto a qué productos se venderán, en qué momento y en qué ubicación. En este contexto, la estimación precisa de la demanda se convierte en un elemento fundamental para la toma de decisiones estratégicas.

Un pronóstico de demanda confiable impacta directamente en:

- La definición de objetivos financieros y de planificación de mercancías
- La determinación de precios óptimos a nivel de SKU (Stock Keeping Unit)
- La gestión eficiente del inventario, buscando maximizar las ventas sin incurrir en sobreinventario

El inventario representa una de las inversiones más significativas para los retailers, ya que suele adquirirse con varios meses de anticipación respecto a su venta. Además del costo de adquisición, existen costos adicionales asociados a la logística, distribución y transporte hacia los puntos de venta.

Una vez que los productos se encuentran en tienda o en centros de distribución, se generan costos de almacenamiento, así como pérdidas por mermas, obsolescencia y estrategias de liquidación. Adicionalmente, el inventario inmoviliza capital de trabajo y ocupa espacio que podría destinarse a productos con mayor rotación, lo que implica un costo de oportunidad considerable.

En consecuencia, diversos indicadores clave de desempeño en el sector retail están directamente relacionados con la gestión del inventario, tales como el retorno sobre la inversión en inventario (GMROI) y los niveles de disponibilidad de producto.

En este sentido, comprender la cantidad óptima de cada SKU en cada tienda y centro de distribución es un problema central para los minoristas modernos.

Subestimar la demanda puede resultar en quiebres de stock, estantes vacíos, pérdida de ventas y deterioro en la experiencia del cliente. Por otro lado, sobreestimar la demanda conduce a excesos de inventario, inmovilización de capital y la necesidad de aplicar descuentos que erosionan los márgenes de ganancia.

Por lo tanto, la estimación precisa de la demanda del consumidor constituye el primer paso para optimizar la gestión del inventario y maximizar la rentabilidad del negocio.

1.3 Presentación del dataset (M5)

El dataset fue proporcionado por *Walmart* y corresponde a las *unidades de ventas* (Sales) de varios de los productos vendidos en Estados Unidos, organizados en series de tiempo. Particularmente, el dataset contiene las ventas de 3,049 productos, clasificados en 3 categorías (Hobbies, Foods y Household) y 7 departamentos. Estas categorías se encuentran de manera desagregada.

Los productos son vendidos a lo largo a través de las 10 tiendas localizadas en 3 estados (California, Texas y Wisconsin).

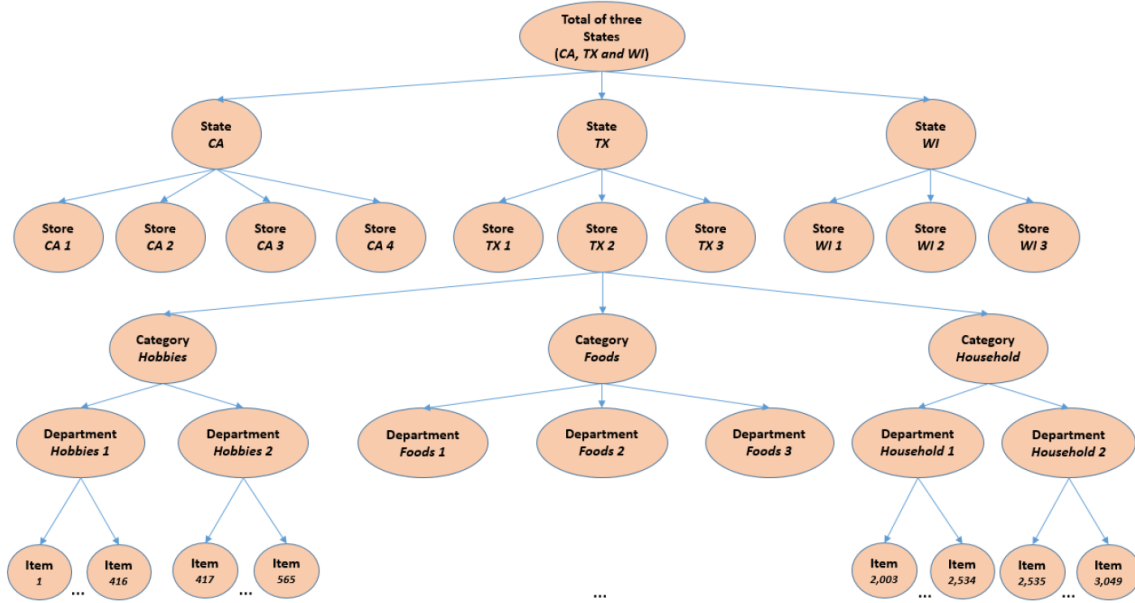


Figure 1: Distribución herárquica de los datos

La data histórica comprende de 2011/01/29 a 2026/06/19 la cual no contiene los 28 días.

El conjunto M5 consiste en los siguientes 3 archivos:

1.3.1 Archivo: calendar.csv

- **date:** Fecha en formato YYYY-MM-DD.
- **wm_yr_wk:** Identificador de la semana a la que pertenece la fecha.
- **weekday:** Nombre del día de la semana (e.g., Saturday, Sunday, ..., Friday).
- **wday:** Identificador numérico del día de la semana, comenzando en sábado.
- **month:** Mes correspondiente a la fecha.
- **year:** Año correspondiente a la fecha.
- **event_name_1:** Nombre del evento asociado a la fecha (si existe).
- **event_type_1:** Tipo del evento asociado (si existe).
- **event_name_2:** Nombre de un segundo evento asociado (si existe).
- **event_type_2:** Tipo de un segundo evento asociado (si existe).
- **snap_CA, snap_TX, snap_WI:** Variables binarias que indican si las tiendas en California (CA), Texas (TX) o Wisconsin (WI) permiten compras mediante SNAP en la fecha analizada. Un valor de 1 indica que estas compras están permitidas.

1.3.2 Archivo: `sell_prices.csv`

Este archivo contiene información sobre los precios de los productos por tienda y semana.

- **store_id**: Identificador de la tienda donde se vende el producto.
- **item_id**: Identificador del producto.
- **wm_yr_wk**: Identificador de la semana.
- **sell_price**: Precio del producto para una combinación específica de tienda y semana. El precio se reporta a nivel semanal (promedio de los siete días). La ausencia de precio indica que el producto no fue vendido en dicha semana. Aunque el precio se mantiene constante dentro de cada semana, puede variar a lo largo del tiempo.

1.3.3 Archivo: `sales_train.csv`

Este archivo contiene el historial de ventas diarias por producto y tienda.

- **item_id**: Identificador del producto.
- **dept_id**: Identificador del departamento al que pertenece el producto.
- **cat_id**: Identificador de la categoría del producto.
- **store_id**: Identificador de la tienda donde se vende el producto.
- **state_id**: Estado donde se localiza la tienda.
- **d_1, d_2, ..., d_1941**: Número de unidades vendidas en el día i , donde d_1 corresponde al 29 de enero de 2011.

1.4 Objetivo del proyecto

El objetivo principal de este proyecto es desarrollar y evaluar modelos de pronóstico de demanda aplicados al conjunto de datos de la competencia *M5 Forecasting*, con el fin de predecir las ventas diarias de múltiples productos a nivel tienda durante un horizonte de 28 días incluyendo variables exógenas como días festivos y días de la semana, pues las compras cambian durante fines de semana, eventos deportivos como el Superbowl o en días donde hay *SNAP* (Supplement Nutrition Assistance Program). De esta manera, obtener una predicción lo más parecida a la realidad.

De manera específica, se busca:

- Implementar un flujo completo de análisis de datos que incluya la limpieza, transformación y exploración del dataset.
- Diseñar e incorporar técnicas de *feature engineering* orientadas a capturar patrones temporales, efectos de calendario, precios y relaciones jerárquicas entre productos.
- Desarrollar y comparar distintos enfoques de modelado, incluyendo modelos basados en árboles de decisión (e.g., XGBoost) y modelos de aprendizaje profundo (e.g., LSTM), con el propósito de analizar sus fortalezas y limitaciones en problemas de series de tiempo.
- Evaluar el desempeño de los modelos utilizando esquemas de validación temporal y métricas apropiadas para el problema de forecasting.
- Analizar los resultados obtenidos para generar conclusiones sobre las variables más relevantes y las estrategias de modelado más efectivas en el contexto del problema.

En conjunto, este proyecto busca no solo obtener predicciones precisas, sino también comprender los factores que influyen en la demanda y las implicaciones de las decisiones de modelado en escenarios reales de retail.

2 Descripción del problema

2.1 Exigencia Computacional

El primer reto del proyecto es la escala del conjunto de datos. Después de realizar las uniones entre las distintas fuentes (ventas, calendario y precios), el dataset resultante alcanza aproximadamente 58,327,370 registros, lo que implica una alta demanda tanto de memoria como de capacidad de procesamiento.

Trabajar con este volumen de información en memoria (por ejemplo, utilizando únicamente **pandas**) puede provocar saturación de la memoria RAM, tiempos de cómputo elevados e incluso fallos en la ejecución. Este problema se intensifica durante etapas como el *feature engineering*, donde se generan múltiples variables adicionales, y durante la validación de modelos.

Para mitigar esta limitación, se optó por utilizar **DuckDB**, un motor de base de datos analítico en memoria optimizado para consultas sobre grandes volúmenes de datos. Esta herramienta permite ejecutar operaciones tipo SQL directamente sobre archivos en disco (por ejemplo, formato **Parquet**), reduciendo significativamente la necesidad de cargar los datos completos en memoria.

Adicionalmente, se implementaron las siguientes estrategias:

- Almacenamiento intermedio en formato **Parquet**, debido a su compresión eficiente y rapidez de lectura.
- Procesamiento por etapas, evitando mantener múltiples copias del dataset en memoria.
- Selección de columnas relevantes para cada fase del pipeline, reduciendo la dimensionalidad del problema.

Estas decisiones permiten escalar el procesamiento de datos de manera eficiente.

Además, se consideró que no todas los años o datos históricos son necesarios, por lo que se decidió acortar la ventana de tiempo a 2 años.

2.2 Naturaleza del Problema

El objetivo consiste en predecir la demanda diaria por SKU para un horizonte de $H = 28$ días hacia adelante. Formalmente, para cada serie temporal indexada por i , se busca aproximar:

$$\hat{\mathbf{y}}_{t+1:t+H}^{(i)} = E \left[\mathbf{Y}_{t+1:t+H}^{(i)} \mid \mathcal{F}_t^{(i)} \right]$$

donde $\mathcal{F}_t^{(i)}$ representa la información disponible hasta el tiempo t .

Esto define un problema de **pronóstico multi-output**, en el cual cada observación genera un vector de salida:

$$\mathbf{y} = (y_{t+1}, y_{t+2}, \dots, y_{t+H}) \in R^H$$

La forma en que se modela esta función depende de la estructura algebraica del modelo utilizado.

2.2.1 Enfoque con XGBoost

XGBoost es un modelo aditivo de árboles que aproxima funciones de la forma:

$$\hat{y} = \phi(x) = \sum_{m=1}^M f_m(x), \quad f_m \in \mathcal{F}$$

donde $x \in R^d$ es un vector de características.

Dado que XGBoost no modela dependencias temporales de forma explícita, el problema secuencial original:

$$\mathbf{y}_{t+1:t+H} = F(x_{1:t})$$

se transforma en un problema tabular mediante ingeniería de variables:

$$x_t = \psi(x_{1:t}) \in R^d$$

donde $\psi(\cdot)$ incluye retardos (*lags*), estadísticas móviles y variables exógenas. Para abordar el problema multi-output, se emplea el esquema:

$$\hat{y}_{t+h} = f^{(h)}(x_t), \quad h = 1, \dots, H$$

lo que implica entrenar H modelos independientes. En la práctica, esto se implementa mediante el `MultiOutputRegressor` de *Scikit-Learn*, el cual descompone el problema vectorial en múltiples problemas de regresión escalar:

$$\hat{\mathbf{y}} = \left(f^{(1)}(x), \dots, f^{(H)}(x) \right)$$

Este enfoque permite reutilizar modelos univariados, aunque ignora dependencias entre horizontes.

2.2.2 Enfoque con LSTM

A diferencia de XGBoost, los modelos LSTM están diseñados para trabajar directamente sobre secuencias. Matemáticamente, un LSTM define un sistema dinámico:

$$(h_t, c_t) = F_\theta(x_t, h_{t-1}, c_{t-1})$$

donde:

- $x_t \in R^d$ es el vector de entrada en el tiempo t
- h_t es el estado oculto
- c_t es el estado de memoria

Las ecuaciones internas del LSTM están dadas por:

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

La predicción multi-output se obtiene como:

$$\hat{\mathbf{y}}_{t+1:t+H} = G(h_t)$$

donde $G : R^k \rightarrow R^H$ es una función de salida (por ejemplo, una capa densa).

2.2.3 Transformación de datos: de tabular a secuencial

Para que el modelo LSTM pueda aprender dependencias temporales, es necesario transformar los datos tabulares en secuencias. Esto se logra mediante una transformación de ventanas deslizantes:

$$\mathcal{T} : \{x_t\}_{t=1}^T \rightarrow \{(X^{(i)}, y^{(i)})\}_{i=1}^N$$

donde:

$$\begin{aligned} X^{(i)} &= (x_i, x_{i+1}, \dots, x_{i+L-1}) \in R^{L \times d} \\ y^{(i)} &= (y_{i+L}, \dots, y_{i+L+H-1}) \in R^H \end{aligned}$$

Esta transformación permite aproximar:

$$\hat{\mathbf{y}}_{t+1:t+H} = E[\mathbf{Y}_{t+1:t+H} \mid X_{t-L+1:t}]$$

En este contexto:

- L es la longitud de la ventana de entrada (*input length*)
- H es el horizonte de predicción

A diferencia del enfoque tabular, donde la historia se resume en un vector de características, aquí se preserva explícitamente la estructura temporal.

2.2.4 Comparación de enfoques

Desde un punto de vista funcional:

- XGBoost aproxima funciones de la forma:

$$f : R^d \rightarrow R$$

- LSTM aproxima funciones sobre secuencias:

$$G : (R^d)^L \rightarrow R^H$$

Esto implica que:

- XGBoost requiere transformar explícitamente la información temporal en variables.
- LSTM modela directamente la dinámica temporal a través de su estado interno.

En conjunto, ambos enfoques representan estrategias complementarias: uno basado en ingeniería de características sobre datos tabulares y otro en el aprendizaje directo de dependencias temporales a partir de secuencias.

2.3 Nivel de granularidad

El problema de pronóstico en la competencia *M5 Forecasting* se aborda a un nivel de granularidad fina, ya que las predicciones se realizan a nivel de combinación *producto-tienda* (`item_id`, `store_id`) y con frecuencia diaria.

Esto implica que cada serie de tiempo corresponde a las ventas diarias de un SKU específico en una tienda particular, lo que da lugar a un total de 30,490 series individuales. Cada una de estas series presenta comportamientos heterogéneos, incluyendo distintos niveles de demanda, estacionalidad, intermitencia y sensibilidad a eventos externos.

Adicionalmente, los datos poseen una estructura jerárquica, en la cual las series individuales pueden agregarse en distintos niveles, tales como categoría (`cat_id`), departamento (`dept_id`), tienda (`store_id`) y estado (`state_id`). Esta jerarquía introduce dependencias entre las series, lo que incrementa la complejidad del problema, ya que las predicciones deben ser coherentes a través de los distintos niveles de agregación.

El alto nivel de granularidad, combinado con la dimensionalidad del problema, genera desafíos importantes tanto en términos de modelado como de capacidad computacional, requiriendo enfoques que sean capaces de capturar patrones locales sin perder la estructura global de los datos.

2.4 Análisis Exploratorio de Datos (EDA)

El análisis exploratorio de los datos permite identificar patrones relevantes en las series de tiempo, así como comprender la estructura temporal y jerárquica del problema.

En primer lugar, se analizó el comportamiento agregado de las ventas por estado. Como se observa en la Figura 2, las series presentan dinámicas similares a lo largo del tiempo, lo que sugiere la existencia de factores comunes que afectan la demanda en las distintas regiones.

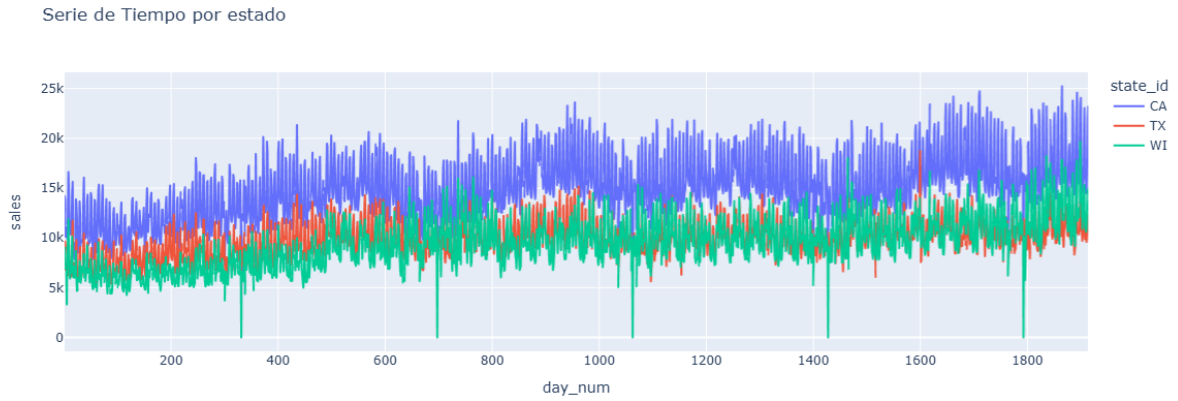


Figure 2: Series de ventas agregadas por estado

Este comportamiento fue corroborado mediante el cálculo de la matriz de correlación entre estados (Figura 3), donde se observan altos niveles de correlación. Esto indica que, a pesar de las diferencias geográficas, los patrones de consumo mantienen una estructura similar, posiblemente influenciada por factores macroeconómicos o estacionales.

Correlación entre estados (basada en patrones de compra de departamento)

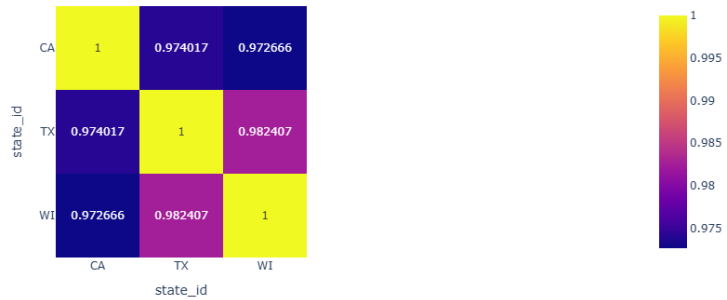


Figure 3: Correlación entre estados basada en patrones de ventas

Posteriormente, se analizó el impacto de variables exógenas provenientes del calendario, tales como eventos (festividades, eventos deportivos, etc.) y días con beneficios SNAP. Estas variables pueden influir en el comportamiento de la demanda al introducir cambios en los patrones de compra.

Como se muestra en la Figura 4, si bien existe un impacto observable en ciertos periodos, este no es consistentemente pronunciado a nivel agregado, lo que sugiere que su efecto podría ser más relevante a niveles de mayor granularidad o en categorías específicas de productos.

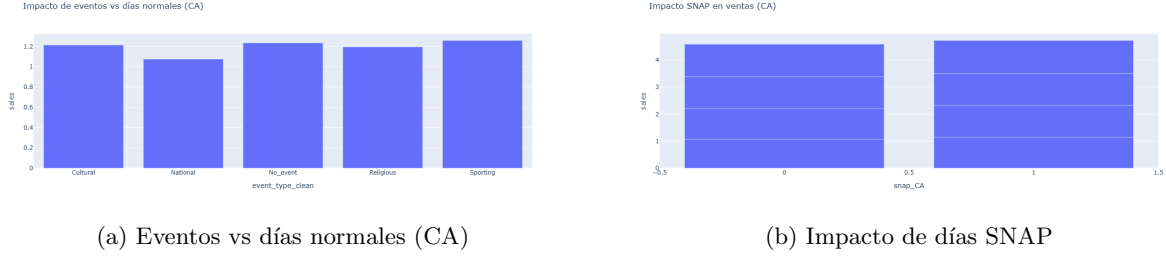


Figure 4: Impacto de variables exógenas en la demanda

En cuanto a la estructura temporal, se identificó una marcada estacionalidad semanal, como se observa en la Figura 5. En particular, los fines de semana presentan sistemáticamente mayores niveles de demanda, lo cual es consistente con patrones de consumo en retail.

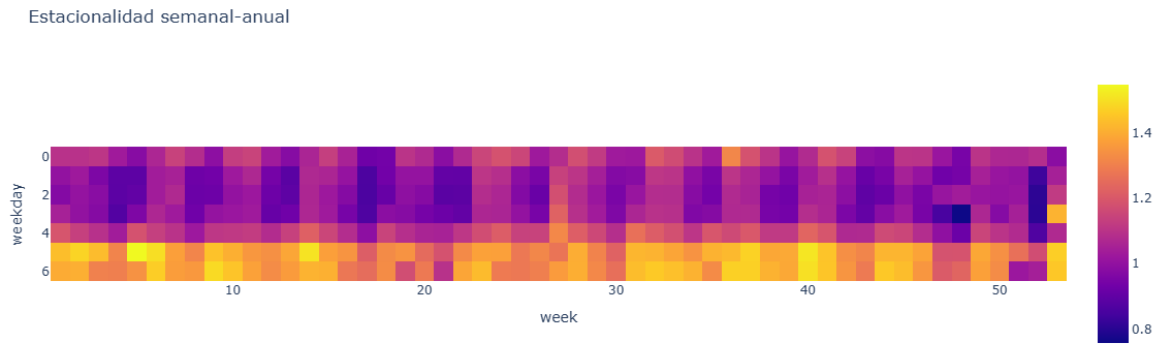
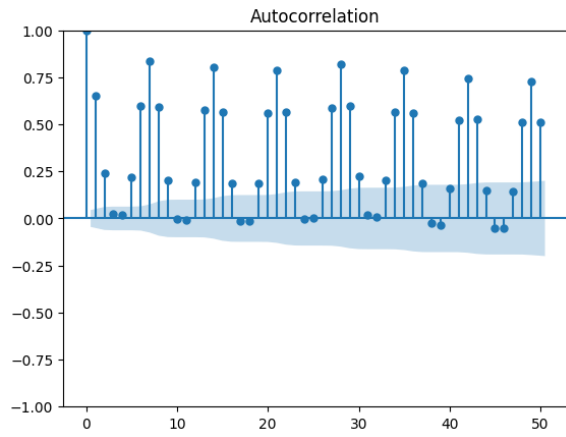


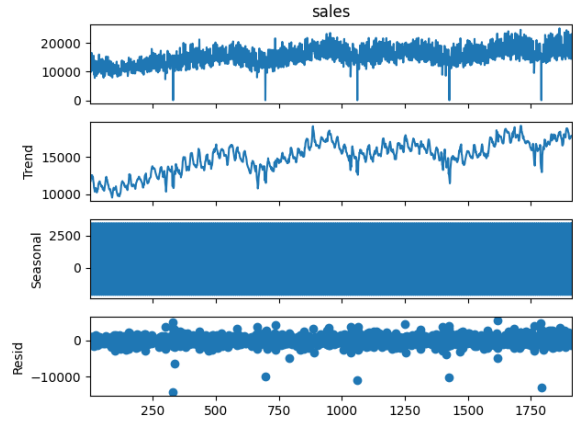
Figure 5: Estacionalidad semanal de la demanda

Este comportamiento se confirma mediante el análisis de autocorrelación, donde se observan picos significativos en múltiplos de 7 días, evidenciando una fuerte dependencia temporal. Asimismo, la presencia de autocorrelación en rezagos cortos indica memoria de corto plazo en la serie, lo que sugiere la pertinencia de modelos capaces de capturar dependencias temporales.

Adicionalmente, la descomposición de la serie temporal (Figura 6) revela una tendencia creciente en el tiempo junto con una componente estacional bien definida. No obstante, los residuos presentan alta variabilidad y la presencia de valores atípicos, lo que indica la existencia de factores no modelados, como promociones o eventos específicos.



(a) Función de autocorrelación (ACF)



(b) Descomposición de la serie temporal

Figure 6: Análisis de dependencia temporal y descomposición

En conjunto, estos hallazgos evidencian que la demanda presenta una estructura compleja caracterizada por estacionalidad, dependencia temporal y efectos exógenos, lo que justifica la incorporación de variables derivadas en el proceso de *feature engineering*.

2.5 Feature Engineering

A partir de los hallazgos obtenidos en el análisis exploratorio de datos, se diseñó un conjunto de variables derivadas con el objetivo de capturar la estructura temporal, la estacionalidad y los efectos exógenos presentes en la demanda.

Dado que se utilizaron dos enfoques de modelado distintos (XGBoost y LSTM), la estrategia de *feature engineering* difiere en función de la capacidad de cada modelo para capturar dependencias temporales.

2.5.1 Ingeniería de características para XGBoost

En el caso de XGBoost, al tratarse de un modelo basado en árboles que no captura dependencias temporales de forma explícita, fue necesario incorporar manualmente información histórica mediante variables derivadas.

Variables de rezago (lags) Motivado por la fuerte estacionalidad semanal identificada en el EDA, se construyeron rezagos en múltiplos de 7 días (28, 35, 42 y 56), permitiendo al modelo capturar patrones recurrentes en la demanda.

Estadísticas móviles (rolling features) Se generaron estadísticas móviles sobre ventanas de 28, 35 y 42 días, incluyendo media, desviación estándar, máximos y mínimos. Estas variables permiten capturar tendencias locales, variabilidad y comportamiento reciente de la serie.

Diferencias temporales Se incluyeron diferencias entre rezagos (e.g., $lag_{28} - lag_{35}$) con el fin de capturar cambios en la tendencia y posibles aceleraciones o desaceleraciones en la demanda.

Variables relacionadas con precios Considerando que el precio es un factor determinante en la demanda, se incorporaron variables como:

- Precio rezagado
- Cambio porcentual del precio
- Precio normalizado respecto a máximos históricos
- Volatilidad del precio

Variables exógenas Se integraron variables provenientes del calendario, tales como eventos y días SNAP. Además, se construyeron variables combinadas (e.g., `snap_event`) para capturar efectos conjuntos.

Codificación temporal Dado el comportamiento cíclico identificado en el EDA, se aplicó codificación sinusoidal a variables como día de la semana y mes, permitiendo representar la periodicidad de manera continua.

Agregaciones jerárquicas Se generaron variables agregadas a nivel de tienda, categoría y departamento, con el objetivo de capturar información global que complementa el comportamiento individual de cada serie.

Target encoding dinámico Para variables categóricas, se implementó un esquema de *target encoding* que respeta la temporalidad, evitando fuga de información mediante el uso de medias acumuladas con rezago.

En conjunto, estas transformaciones permiten que XGBoost incorpore explícitamente información temporal y contextual que de otra forma no podría modelar.

2.5.2 Ingeniería de características para LSTM

A diferencia de XGBoost, los modelos LSTM están diseñados para capturar dependencias temporales de manera implícita a través de su arquitectura recurrente. Por lo tanto, la estrategia de *feature engineering* se enfocó en estructurar adecuadamente los datos en forma de secuencias, en lugar de generar múltiples variables derivadas.

Estructuración secuencial Se construyeron ventanas deslizantes de longitud fija (56 días) como entrada del modelo, con un horizonte de predicción de 28 días. Este enfoque permite que la red aprenda directamente las dependencias temporales a partir de la secuencia de observaciones.

Variables numéricas Se incluyeron variables como ventas, precios, indicadores SNAP y variables cíclicas (senos y cosenos), proporcionando información tanto endógena como exógena al modelo.

Variables categóricas y embeddings Las variables categóricas (e.g., `item_id`, `store_id`) fueron codificadas mediante *label encoding* y posteriormente transformadas en *embeddings*, permitiendo al modelo aprender representaciones densas de estas categorías.

Variables de precio y eventos Se incluyeron algunas transformaciones básicas, como rezagos de precio y cambios porcentuales, así como indicadores de eventos, con el fin de complementar la información temporal aprendida por la red.

Clustering de series Se incorporó una variable adicional derivada de técnicas de clustering, con el objetivo de agrupar series con comportamientos similares y facilitar el aprendizaje de patrones comunes.

2.5.3 Comparación entre enfoques

La principal diferencia entre ambos enfoques radica en la forma en que se modela la dependencia temporal:

- En XGBoost, las dependencias temporales deben ser explícitamente codificadas mediante variables como rezagos y estadísticas móviles.
- En LSTM, estas dependencias son aprendidas directamente a partir de la secuencia de datos, reduciendo la necesidad de generar manualmente múltiples transformaciones.

No obstante, mientras que XGBoost depende en gran medida de la calidad del *feature engineering*, los modelos LSTM requieren una adecuada estructuración de los datos y un mayor costo computacional para su entrenamiento.

En este sentido, ambos enfoques representan dos paradigmas complementarios en el modelado de series de tiempo: uno basado en la ingeniería explícita de variables y otro en el aprendizaje automático de representaciones temporales.

3 Modelos

Como se abordó en la sección 2.2, cada modelo tiene una naturaleza algebraica diferente, por lo que cada uno tiene una arquitectura y espacio hiperparametral muy diferentes.

3.1 Justificación

XGBoost es un modelo muy poderoso y versátil. Su rango de aplicaciones es vasto y aunque no fue diseñado para series de tiempo, es muy utilizado por su capacidad de aprender patrones no lineales que modelos tradicionales (*ARIMA*, *SARIMA*, etc.)

3.2 Modelo basado en XGBoost

Con el objetivo de establecer un modelo base robusto para el pronóstico de la demanda, se implementó un modelo de *gradient boosting* utilizando XGBoost. Este enfoque es particularmente adecuado para datos tabulares con alta dimensionalidad y variables derivadas, como las obtenidas en la etapa de *feature engineering*.

Formulación del problema El problema se abordó como una tarea de regresión multi-salida (*multi-output regression*), donde el objetivo es predecir simultáneamente las ventas para un horizonte de 28 días. Para ello, se construyeron 28 variables objetivo ($target_1, \dots, target_{28}$), cada una correspondiente a un día futuro.

Dado que XGBoost no soporta de forma nativa salidas múltiples, se utilizó un esquema de envoltura mediante `MultiOutputRegressor`, el cual entrena un modelo independiente por cada horizonte de predicción.

Esquema de partición Se empleó una estrategia de validación temporal, respetando la estructura secuencial de los datos. En particular, se definió un punto de corte (*split date*) como los últimos 28 días disponibles en el dataset:

- Conjunto de entrenamiento: observaciones con fecha anterior o igual al punto de corte
- Conjunto de prueba: últimos 28 días

Este enfoque evita la fuga de información (*data leakage*) y simula un escenario real de predicción.

Variables de entrada El modelo utiliza un conjunto de variables compuesto por:

- Variables continuas: rezagos, estadísticas móviles, variables de precio y codificaciones temporales
- Variables categóricas transformadas mediante *target encoding dinámico*
- Variables exógenas: indicadores de eventos y SNAP

Esta combinación permite incorporar tanto información histórica como factores externos que influyen en la demanda.

Arquitectura del modelo Se utilizó el algoritmo XGBoost con árboles de decisión potenciados mediante *gradient boosting*. La implementación se configuró con el método de histogramas (`tree_method = "hist"`), lo cual mejora la eficiencia computacional en datasets de gran tamaño.

Hiperparámetros Los principales hiperparámetros utilizados fueron:

- `n_estimators = 300`: número de árboles en el ensamble
- `learning_rate = 0.05`: tasa de aprendizaje que controla la contribución de cada árbol
- `max_depth = 8`: profundidad máxima de los árboles, permitiendo capturar interacciones complejas
- `subsample = 0.8`: fracción de observaciones utilizadas por árbol, reduciendo el sobreajuste

- **colsample_bytree = 0.8**: fracción de variables utilizadas por árbol
- **objective = reg:tweedie**: función objetivo adecuada para variables con distribución sesgada y presencia de ceros, como es común en datos de demanda

Entrenamiento El modelo fue entrenado utilizando todas las observaciones disponibles en el conjunto de entrenamiento, ajustando un modelo independiente por cada horizonte de predicción. Posteriormente, se generaron predicciones sobre el conjunto de prueba.

3.3 Modelo basado en LSTM

Con el fin de capturar de manera explícita las dependencias temporales presentes en las series de demanda, se implementó un modelo basado en redes neuronales recurrentes del tipo Long Short-Term Memory (LSTM). Este enfoque permite modelar secuencias de datos y aprender patrones temporales de forma automática, a diferencia de modelos basados en árboles.

Formulación del problema El problema se planteó como una tarea de predicción multi-horizonte, donde el modelo recibe una secuencia de observaciones pasadas y genera como salida un vector con las predicciones de los siguientes 28 días.

A diferencia del enfoque utilizado en XGBoost, en este caso la dependencia temporal no se codifica explícitamente mediante variables derivadas extensivas, sino que es aprendida directamente por la red a partir de la secuencia de entrada.

Transformación de la variable objetivo Con el objetivo de estabilizar la varianza y reducir el impacto de valores extremos, se aplicó una transformación logarítmica a la variable objetivo:

$$y_t = \log(1 + \text{sales}_t)$$

Posteriormente, las predicciones fueron transformadas de vuelta a la escala original mediante la función inversa.

Variables de entrada Se utilizaron dos tipos de variables:

- **Variables numéricas**: incluyen ventas históricas, precios, indicadores SNAP, variables cíclicas (codificadas mediante funciones seno y coseno) y un conjunto reducido de rezagos relevantes (7 y 28 días).
- **Variables categóricas**: `item_id`, `dept_id`, `cat_id`, `store_id` y `state_id`.

Estructuración secuencial Se construyeron secuencias de entrada de longitud fija de 56 días (*input length*), con un horizonte de predicción de 28 días. Para aumentar la cantidad de muestras de entrenamiento, se utilizó una ventana deslizante con un *stride* de 7 días.

Cada muestra está compuesta por:

- Una matriz de variables numéricas de dimensión $56 \times d$
- Un vector de variables categóricas asociadas a la serie
- Un vector objetivo de longitud 28

Codificación de variables categóricas Las variables categóricas fueron transformadas mediante *label encoding* y posteriormente representadas a través de capas de *embeddings*. Esto permite al modelo aprender representaciones densas de alta dimensionalidad, capturando similitudes entre categorías.

El tamaño de los embeddings se definió como una función del número de categorías, con el objetivo de balancear capacidad representacional y costo computacional.

Escalado de variables Las variables numéricas fueron normalizadas mediante *MinMaxScaler*, lo cual es fundamental para garantizar la estabilidad del entrenamiento en redes neuronales.

Arquitectura del modelo El modelo propuesto consta de dos ramas principales:

- **Rama temporal:** compuesta por dos capas LSTM (64 y 32 unidades, respectivamente), encargadas de procesar la secuencia de entrada y capturar dependencias temporales. Se incorporaron capas de *Dropout* para reducir el sobreajuste.
- **Rama categórica:** formada por embeddings independientes para cada variable categórica, cuyos vectores son concatenados para generar una representación conjunta.

Ambas ramas se combinan mediante una capa de concatenación, seguida de capas densas completamente conectadas (128 y 64 neuronas) que permiten aprender interacciones entre variables temporales y categóricas.

Finalmente, la capa de salida genera un vector de dimensión 28 correspondiente al horizonte de predicción.

Entrenamiento El modelo fue entrenado utilizando el optimizador Adam con una tasa de aprendizaje de 5×10^{-4} y función de pérdida basada en error cuadrático medio (MSE).

Se implementaron diversas estrategias para mejorar la generalización:

- **Early stopping:** detiene el entrenamiento cuando la pérdida de validación deja de mejorar
- **Model checkpoint:** guarda el mejor modelo durante el entrenamiento
- **ReduceLROnPlateau:** reduce la tasa de aprendizaje cuando el modelo alcanza un estancamiento

Esquema de partición Se utilizó una validación temporal en dos niveles:

- División entrenamiento/prueba basada en los últimos 28 días del dataset
- División entrenamiento/validación sobre el conjunto de entrenamiento (90% / 10%)

Este enfoque asegura que el modelo sea evaluado en datos no vistos y respeta la estructura temporal del problema.

3.4 Evaluación

La evaluación de los modelos se realizó considerando tanto métricas tradicionales de regresión como métricas específicas del contexto de series de tiempo en retail.

Métricas utilizadas Se emplearon las siguientes métricas:

- **MAE (Mean Absolute Error):** mide el error absoluto promedio entre las predicciones y los valores reales. Es una métrica robusta e interpretable en el contexto del negocio, ya que representa el error promedio en unidades vendidas.
- **RMSE (Root Mean Squared Error):** penaliza en mayor medida los errores grandes, siendo útil para identificar desviaciones significativas en la predicción.
- **MAPE (Mean Absolute Percentage Error):** mide el error relativo; sin embargo, esta métrica presenta limitaciones en este contexto debido a la presencia de valores cercanos a cero en la demanda, lo que puede generar valores inestables.
- **RMSSE (Root Mean Squared Scaled Error):** métrica utilizada en la competencia M5, que normaliza el error en función de la variabilidad histórica de cada serie, permitiendo comparaciones entre series con distintas escalas.

Esquema de evaluación La evaluación se realizó sobre un conjunto de prueba correspondiente a los últimos 28 días disponibles, simulando un escenario real de predicción futura. En el caso del modelo LSTM, las predicciones fueron transformadas nuevamente a la escala original antes del cálculo de métricas.

Consideraciones sobre las métricas Es importante destacar que:

- El MAE es la métrica principal de comparación, debido a su interpretabilidad y estabilidad.
- El MAPE no resulta adecuado en este contexto debido a la alta frecuencia de valores cero o cercanos a cero en la demanda, lo que explica los valores extremadamente altos observados.
- El RMSSE permite evaluar el desempeño relativo considerando la dinámica propia de cada serie, alineándose con el enfoque de la competencia M5.

3.5 Resultados

Los resultados obtenidos para cada modelo se presentan a continuación:

Modelo XGBoost

- RMSE: 2.19
- MAE: 1.03
- RMSSE: (valor calculado)

Modelo LSTM

- RMSE: 2.49
- MAE: 1.02

Análisis comparativo En términos de MAE, ambos modelos presentan un desempeño muy similar, con una ligera ventaja para el modelo LSTM. Esto sugiere que ambos enfoques son capaces de capturar adecuadamente la dinámica general de la demanda.

No obstante, al analizar el RMSE, el modelo XGBoost muestra un mejor desempeño, lo que indica una menor presencia de errores grandes en comparación con LSTM. Esto puede atribuirse a la robustez de los modelos basados en árboles frente a valores atípicos.

Desde una perspectiva metodológica, los resultados reflejan las diferencias entre ambos enfoques:

- **XGBoost** logra un desempeño competitivo gracias a un extenso proceso de *feature engineering*, que permite incorporar explícitamente la estructura temporal y los efectos exógenos.
- **LSTM**, por su parte, obtiene resultados comparables con un menor nivel de ingeniería manual de variables, aprovechando su capacidad para aprender representaciones temporales directamente a partir de los datos.

Interpretación en el contexto del negocio Un MAE cercano a 1 implica que, en promedio, el error del modelo es de aproximadamente una unidad vendida por día y por serie. Considerando el nivel de granularidad del problema (producto-tienda), este nivel de error puede considerarse razonable.

Limitaciones A pesar de los resultados obtenidos, existen algunas limitaciones relevantes:

- La evaluación se realizó a nivel agregado, sin considerar la estructura jerárquica completa utilizada en la métrica oficial del M5 (WRMSSE).
- El modelo LSTM podría beneficiarse de una mayor optimización de hiperparámetros y arquitectura.
- La inclusión de variables adicionales, como promociones o información externa, podría mejorar el desempeño de ambos modelos.

Conclusión de resultados En conjunto, los resultados muestran que ambos enfoques son viables para el problema de pronóstico de demanda. Sin embargo, el modelo XGBoost ofrece una solución más eficiente computacionalmente y con menor sensibilidad a errores extremos, mientras que el modelo LSTM presenta un enfoque más flexible y escalable para capturar dependencias temporales complejas.

4 Conclusiones

En este proyecto se abordó el problema de pronóstico de demanda utilizando el dataset de la competencia *M5 Forecasting*, el cual representa un escenario realista caracterizado por alta dimensionalidad, granularidad fina y una estructura jerárquica compleja.

A partir del análisis exploratorio de datos, se identificaron patrones clave en las series de tiempo, incluyendo una marcada estacionalidad semanal, dependencia temporal de corto plazo y efectos asociados a variables exógenas como eventos de calendario y días SNAP. Estos hallazgos guiaron el diseño de las estrategias de *feature engineering* y la selección de los modelos.

Se implementaron dos enfoques principales de modelado: XGBoost y LSTM, los cuales representan paradigmas distintos en el análisis de series de tiempo. El modelo XGBoost, basado en árboles de decisión, requirió un proceso intensivo de ingeniería de variables para incorporar explícitamente la información temporal y contextual. Por otro lado, el modelo LSTM permitió capturar dependencias temporales de manera implícita mediante el uso de secuencias y embeddings para variables categóricas.

En términos de desempeño, ambos modelos mostraron resultados comparables en la métrica MAE, con un error promedio cercano a una unidad por día y por serie. No obstante, el modelo XGBoost presentó un mejor comportamiento en términos de RMSE, indicando una menor sensibilidad a errores grandes. Esto sugiere que, en este contexto, los modelos basados en árboles continúan siendo altamente competitivos en problemas de datos tabulares.

Desde una perspectiva metodológica, este trabajo evidencia que el éxito en problemas de forecasting depende no solo del modelo utilizado, sino también de la calidad del procesamiento de datos y la ingeniería de características. En particular, el enfoque basado en XGBoost destaca por su eficiencia computacional y robustez, mientras que el modelo LSTM ofrece mayor flexibilidad para capturar relaciones temporales complejas, aunque a un mayor costo computacional.

5 Referencias

References

- [1] Kaggle. *M5 Forecasting - Accuracy*. Disponible en: <https://www.kaggle.com/competitions/m5-forecasting-accuracy/data>
- [2] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. *Neural Computation*, 9(8), 1735–1780.
- [3] Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*.
- [4] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. *Proceedings of the 22nd ACM SIGKDD*.
- [5] Brownlee, J. *Multi-step Time Series Forecasting with XGBoost*. Disponible en: <https://towardsdatascience.com/multi-step-time-series-forecasting-with-xgboost-65d6820bec39>
- [6] *XGBoost for Time Series: You're Gonna Need a Bigger Boat*. Disponible en: <https://towardsdatascience.com/xgboost-for-time-series-youre-gonna-need-a-bigger-boat-9d329efa6814/>
- [7] Akalazu, C. *The Power of Encoding Techniques*. Disponible en: <https://medium.com/@clintonakalazu/unveiling-the-power-of-encoding-techniques-a-comparative-analysis-of-label-enco>
- [8] *Categorical Encoding for Time Series*. Disponible en: <https://pub.towardsai.net/categorical-encoding-for-time-series-embracing-dynamic-and-meaningful-techniques-d5e796c07c8f>
- [9] *Neural Network Embeddings Explained*. Disponible en: <https://medium.com/data-science/neural-network-embeddings-explained-4d028e6f0526>
- [10] *LSTM: How to Teach a Network to Remember the Past*. Disponible en: <https://towardsdatascience.com/lstm-recurrent-neural-networks-how-to-teach-a-network-to-remember-the-pa>
- [11] *N-HiTS: Making Deep Learning for Time Series Forecasting More Efficient*. Disponible en: <https://medium.com/data-science/n-hits-making-deep-learning-for-time-series-forecasting-more-efficient-d00956fc3e93>